

Programmatic access and feedback loops

In an agentic workflow, the model can use tools.

In an agentic workflow, the model can use tools.

Lets us create a **closed-loop** interaction:

In an agentic workflow, the model can use tools.

Lets us create a **closed-loop** interaction:

`generate code -> run tests -> read errors -> edit code -> run tests again`

In an agentic workflow, the model can use tools.

Lets us create a **closed-loop** interaction:

`generate code -> run tests -> read errors -> edit code -> run tests again`

The model is not just writing text, but is acting in a development environment and iterating on its own output.

Example: an agent loop

You can make your own agentic loop with a model API and a simple while loop.

Example: an agent loop

You can make your own agentic loop with a model API and a simple while loop.

```
while not tests_pass():  
    proposed_patch = model.generate_patch(  
        instructions=task,  
        context=current_files(),  
        last_error=test_output(),  
    )  
    apply_patch(proposed_patch)  
    run("python -m unittest discover -s tests")
```

Tool use changes the risk landscape

A chatbot can give you bad code but will sort of just sit there after.

Tool use changes the risk landscape

A chatbot can give you bad code but will sort of just sit there after.

An agent is much more powerful! It can...

Tool use changes the risk landscape

A chatbot can give you bad code but will sort of just sit there after.

An agent is much more powerful! It can...

- edit many files

Tool use changes the risk landscape

A chatbot can give you bad code but will sort of just sit there after.

An agent is much more powerful! It can...

- edit many files
- install dependencies

Tool use changes the risk landscape

A chatbot can give you bad code but will sort of just sit there after.

An agent is much more powerful! It can...

- edit many files
- install dependencies
- run shell commands

Tool use changes the risk landscape

A chatbot can give you bad code but will sort of just sit there after.

An agent is much more powerful! It can...

- edit many files
- install dependencies
- run shell commands
- keep trying a bad approach until you stop it

Tool use changes the risk landscape

A chatbot can give you bad code but will sort of just sit there after.

An agent is much more powerful! It can...

- edit many files
- install dependencies
- run shell commands
- keep trying a bad approach until you stop it
- modify tests to make itself look successful

Tool use changes the risk landscape

A chatbot can give you bad code but will sort of just sit there after.

An agent is much more powerful! It can...

- edit many files
- install dependencies
- run shell commands
- keep trying a bad approach until you stop it
- modify tests to make itself look successful
- delete or overwrite work

Tool use changes the risk landscape

A chatbot can give you bad code but will sort of just sit there after.

An agent is much more powerful! It can...

- edit many files
- install dependencies
- run shell commands
- keep trying a bad approach until you stop it
- modify tests to make itself look successful
- delete or overwrite work
- **seriously** fuck up your codebase, not to mention your machine

Tool use changes the risk landscape

A chatbot can give you bad code but will sort of just sit there after.

An agent is much more powerful! It can...

- edit many files
- install dependencies
- run shell commands
- keep trying a bad approach until you stop it
- modify tests to make itself look successful
- delete or overwrite work
- **seriously** fuck up your codebase, not to mention your machine
- break containment if you're not careful

Tool use changes the risk landscape

A chatbot can give you bad code but will sort of just sit there after.

An agent is much more powerful! It can...

- edit many files
- install dependencies
- run shell commands
- keep trying a bad approach until you stop it
- modify tests to make itself look successful
- delete or overwrite work
- **seriously** fuck up your codebase, not to mention your machine
- break containment if you're not careful

Tool use changes the risk landscape

A chatbot can give you bad code but will sort of just sit there after.

An agent is much more powerful! It can...

- edit many files
- install dependencies
- run shell commands
- keep trying a bad approach until you stop it
- modify tests to make itself look successful
- delete or overwrite work
- **seriously** fuck up your codebase, not to mention your machine
- break containment if you're not careful

More autonomy → stronger guardrails.

- **Sandboxing:** isolate the agent from the rest of the system

Basic safety measures: Sandboxing and permissions

- **Sandboxing:** isolate the agent from the rest of the system
 - sandboxed environments like Docker containers

Basic safety measures: Sandboxing and permissions

- **Sandboxing:** isolate the agent from the rest of the system
 - sandboxed environments like Docker containers
- **Permissions:** restrict the agent's ability to read/write certain files

Basic safety measures: Sandboxing and permissions

- **Sandboxing:** isolate the agent from the rest of the system
 - sandboxed environments like Docker containers
- **Permissions:** restrict the agent's ability to read/write certain files
 - can't edit test suites

Basic safety measures: Sandboxing and permissions

- **Sandboxing:** isolate the agent from the rest of the system
 - sandboxed environments like Docker containers
- **Permissions:** restrict the agent's ability to read/write certain files
 - can't edit test suites
 - can't edit outside the project directory

Basic safety measures: Sandboxing and permissions

- **Sandboxing:** isolate the agent from the rest of the system
 - sandboxed environments like Docker containers
- **Permissions:** restrict the agent's ability to read/write certain files
 - can't edit test suites
 - can't edit outside the project directory
 - can't read keys and other secrets

Basic safety measures: Sandboxing and permissions

- **Sandboxing:** isolate the agent from the rest of the system
 - sandboxed environments like Docker containers
- **Permissions:** restrict the agent's ability to read/write certain files
 - can't edit test suites
 - can't edit outside the project directory
 - can't read keys and other secrets
- **Commit early, commit often**

Basic safety measures: Sandboxing and permissions

- **Sandboxing:** isolate the agent from the rest of the system
 - sandboxed environments like Docker containers
- **Permissions:** restrict the agent's ability to read/write certain files
 - can't edit test suites
 - can't edit outside the project directory
 - can't read keys and other secrets
- **Commit early, commit often**
 - the agent can't rewrite git history

Basic safety measures: Sandboxing and permissions

- **Sandboxing:** isolate the agent from the rest of the system
 - sandboxed environments like Docker containers
- **Permissions:** restrict the agent's ability to read/write certain files
 - can't edit test suites
 - can't edit outside the project directory
 - can't read keys and other secrets
- **Commit early, commit often**
 - the agent can't rewrite git history
- **Test constantly**

Basic safety measures: Sandboxing and permissions

- **Sandboxing:** isolate the agent from the rest of the system
 - sandboxed environments like Docker containers
- **Permissions:** restrict the agent's ability to read/write certain files
 - can't edit test suites
 - can't edit outside the project directory
 - can't read keys and other secrets
- **Commit early, commit often**
 - the agent can't rewrite git history
- **Test constantly**
 - the agent can still make a mess of your repo

Four interaction patterns

Pattern	Typical use
Single-turn generation	Draft one function, regular expression, SQL query, or explanation
Context-aware completion	Fill in code inside an editor using nearby files as context
Programmatic access	Call a model API from Python, as in the Gemini example
Autonomous execution	Let an agent inspect files, edit code, run tests, and revise

Four interaction patterns

Pattern	Typical use
Single-turn generation	Draft one function, regular expression, SQL query, or explanation
Context-aware completion	Fill in code inside an editor using nearby files as context
Programmatic access	Call a model API from Python, as in the Gemini example
Autonomous execution	Let an agent inspect files, edit code, run tests, and revise

These are not strict categories, most current tools blur them.

Pattern 1: single-turn generation

- You ask for something once.

Pattern 1: single-turn generation

- You ask for something once.
- The model answers once.

Pattern 1: single-turn generation

- You ask for something once.
- The model answers once.

Pattern 1: single-turn generation

- You ask for something once.
- The model answers once.

Good for:

Pattern 1: single-turn generation

- You ask for something once.
- The model answers once.

Good for:

- small examples

Pattern 1: single-turn generation

- You ask for something once.
- The model answers once.

Good for:

- small examples
- explanations

Pattern 1: single-turn generation

- You ask for something once.
- The model answers once.

Good for:

- small examples
- explanations
- scaffolding

Pattern 1: single-turn generation

- You ask for something once.
- The model answers once.

Good for:

- small examples
- explanations
- scaffolding
- syntax you half-remember

Pattern 1: single-turn generation

- You ask for something once.
- The model answers once.

Good for:

- small examples
- explanations
- scaffolding
- syntax you half-remember

Pattern 1: single-turn generation

- You ask for something once.
- The model answers once.

Good for:

- small examples
- explanations
- scaffolding
- syntax you half-remember

Bad for:

Pattern 1: single-turn generation

- You ask for something once.
- The model answers once.

Good for:

- small examples
- explanations
- scaffolding
- syntax you half-remember

Bad for:

- ambiguous scientific logic

Pattern 1: single-turn generation

- You ask for something once.
- The model answers once.

Good for:

- small examples
- explanations
- scaffolding
- syntax you half-remember

Bad for:

- ambiguous scientific logic
- multi-file refactors

Pattern 1: single-turn generation

- You ask for something once.
- The model answers once.

Good for:

- small examples
- explanations
- scaffolding
- syntax you half-remember

Bad for:

- ambiguous scientific logic
- multi-file refactors
- tasks where correctness is not easy to check

Pattern 2: context-aware completion

- The model reads nearby code and predicts what should come next.

Pattern 2: context-aware completion

- The model reads nearby code and predicts what should come next.
- This can feel like pair programming.

Pattern 2: context-aware completion

- The model reads nearby code and predicts what should come next.
- This can feel like pair programming.

Pattern 2: context-aware completion

- The model reads nearby code and predicts what should come next.
- This can feel like pair programming.

It is useful when:

Pattern 2: context-aware completion

- The model reads nearby code and predicts what should come next.
- This can feel like pair programming.

It is useful when:

- naming conventions are already clear

Pattern 2: context-aware completion

- The model reads nearby code and predicts what should come next.
- This can feel like pair programming.

It is useful when:

- naming conventions are already clear
- types and function signatures are nearby

Pattern 2: context-aware completion

- The model reads nearby code and predicts what should come next.
- This can feel like pair programming.

It is useful when:

- naming conventions are already clear
- types and function signatures are nearby
- the next step is local and obvious

Pattern 2: context-aware completion

- The model reads nearby code and predicts what should come next.
- This can feel like pair programming.

It is useful when:

- naming conventions are already clear
- types and function signatures are nearby
- the next step is local and obvious

Pattern 2: context-aware completion

- The model reads nearby code and predicts what should come next.
- This can feel like pair programming.

It is useful when:

- naming conventions are already clear
- types and function signatures are nearby
- the next step is local and obvious

It is risky when the local pattern is wrong – AI is **great** at imitating bad examples. It's also **wildly annoying** when it keeps trying to autocomplete based on some misunderstanding.

Pattern 3: programmatic access

You call the model from code. That lets you:

Pattern 3: programmatic access

You call the model from code. That lets you:

- repeat prompts

Pattern 3: programmatic access

You call the model from code. That lets you:

- repeat prompts
- compare outputs

Pattern 3: programmatic access

You call the model from code. That lets you:

- repeat prompts
- compare outputs
- save responses

Pattern 3: programmatic access

You call the model from code. That lets you:

- repeat prompts
- compare outputs
- save responses
- build retry loops

Pattern 3: programmatic access

You call the model from code. That lets you:

- repeat prompts
- compare outputs
- save responses
- build retry loops
- combine model output with automated checks

Pattern 3: programmatic access

You call the model from code. That lets you:

- repeat prompts
- compare outputs
- save responses
- build retry loops
- combine model output with automated checks

Pattern 3: programmatic access

You call the model from code. That lets you:

- repeat prompts
- compare outputs
- save responses
- build retry loops
- combine model output with automated checks

A model API is still just an API!

Pattern 4: autonomous execution

- You give the agent a goal.

Pattern 4: autonomous execution

- You give the agent a goal.
- It explores, edits, runs commands, and reports back.

Pattern 4: autonomous execution

- You give the agent a goal.
- It explores, edits, runs commands, and reports back.

Pattern 4: autonomous execution

- You give the agent a goal.
- It explores, edits, runs commands, and reports back.

Good for:

Pattern 4: autonomous execution

- You give the agent a goal.
- It explores, edits, runs commands, and reports back.

Good for:

- well-tested refactors

Pattern 4: autonomous execution

- You give the agent a goal.
- It explores, edits, runs commands, and reports back.

Good for:

- well-tested refactors
- mechanical code changes

Pattern 4: autonomous execution

- You give the agent a goal.
- It explores, edits, runs commands, and reports back.

Good for:

- well-tested refactors
- mechanical code changes
- repetitive cleanup

Pattern 4: autonomous execution

- You give the agent a goal.
- It explores, edits, runs commands, and reports back.

Good for:

- well-tested refactors
- mechanical code changes
- repetitive cleanup
- generating documentation from existing structure

Pattern 4: autonomous execution

- You give the agent a goal.
- It explores, edits, runs commands, and reports back.

Good for:

- well-tested refactors
- mechanical code changes
- repetitive cleanup
- generating documentation from existing structure

Pattern 4: autonomous execution

- You give the agent a goal.
- It explores, edits, runs commands, and reports back.

Good for:

- well-tested refactors
- mechanical code changes
- repetitive cleanup
- generating documentation from existing structure

Risky for:

Pattern 4: autonomous execution

- You give the agent a goal.
- It explores, edits, runs commands, and reports back.

Good for:

- well-tested refactors
- mechanical code changes
- repetitive cleanup
- generating documentation from existing structure

Risky for:

- weakly specified scientific methods

Pattern 4: autonomous execution

- You give the agent a goal.
- It explores, edits, runs commands, and reports back.

Good for:

- well-tested refactors
- mechanical code changes
- repetitive cleanup
- generating documentation from existing structure

Risky for:

- weakly specified scientific methods
- code with no tests

Pattern 4: autonomous execution

- You give the agent a goal.
- It explores, edits, runs commands, and reports back.

Good for:

- well-tested refactors
- mechanical code changes
- repetitive cleanup
- generating documentation from existing structure

Risky for:

- weakly specified scientific methods
- code with no tests
- sensitive data

Four interaction patterns

Pattern	Typical use
Single-turn generation	Draft one function, regular expression, SQL query, or explanation
Context-aware completion	Fill in code inside an editor using nearby files as context
Programmatic access	Call a model API from Python, as in the Gemini example
Autonomous execution	Let an agent inspect files, edit code, run tests, and revise

AI does not make programming expertise obsolete, but changes the balance of which skills matter.

Shift in skill requirements

AI does not make programming expertise obsolete, but changes the balance of which skills matter.

Less important:

Shift in skill requirements

AI does not make programming expertise obsolete, but changes the balance of which skills matter.

Less important:

- remembering every syntax detail

Shift in skill requirements

AI does not make programming expertise obsolete, but changes the balance of which skills matter.

Less important:

- remembering every syntax detail
- writing boilerplate by hand

Shift in skill requirements

AI does not make programming expertise obsolete, but changes the balance of which skills matter.

Less important:

- remembering every syntax detail
- writing boilerplate by hand
- starting from a blank file

Shift in skill requirements

AI does not make programming expertise obsolete, but changes the balance of which skills matter.

Less important:

- remembering every syntax detail
- writing boilerplate by hand
- starting from a blank file

Shift in skill requirements

AI does not make programming expertise obsolete, but changes the balance of which skills matter.

Less important:

- remembering every syntax detail
- writing boilerplate by hand
- starting from a blank file

More important:

Shift in skill requirements

AI does not make programming expertise obsolete, but changes the balance of which skills matter.

Less important:

- remembering every syntax detail
- writing boilerplate by hand
- starting from a blank file

More important:

- decomposing the problem

Shift in skill requirements

AI does not make programming expertise obsolete, but changes the balance of which skills matter.

Less important:

- remembering every syntax detail
- writing boilerplate by hand
- starting from a blank file

More important:

- decomposing the problem
- defining success

Shift in skill requirements

AI does not make programming expertise obsolete, but changes the balance of which skills matter.

Less important:

- remembering every syntax detail
- writing boilerplate by hand
- starting from a blank file

More important:

- decomposing the problem
- defining success
- reviewing code

Shift in skill requirements

AI does not make programming expertise obsolete, but changes the balance of which skills matter.

Less important:

- remembering every syntax detail
- writing boilerplate by hand
- starting from a blank file

More important:

- decomposing the problem
- defining success
- reviewing code
- testing behavior

Shift in skill requirements

AI does not make programming expertise obsolete, but changes the balance of which skills matter.

Less important:

- remembering every syntax detail
- writing boilerplate by hand
- starting from a blank file

More important:

- decomposing the problem
- defining success
- reviewing code
- testing behavior
- recognizing when the model is going in circles

What we are building

A Python script that:

What we are building

A Python script that:

1. reads a task description from `prompt.txt`

What we are building

A Python script that:

1. reads a task description from `prompt.txt`
2. calls Gemini and receives a Python file

What we are building

A Python script that:

1. reads a task description from `prompt.txt`
2. calls Gemini and receives a Python file
3. writes the file to disk

What we are building

A Python script that:

1. reads a task description from `prompt.txt`
2. calls Gemini and receives a Python file
3. writes the file to disk
4. runs `unittest`

What we are building

A Python script that:

1. reads a task description from `prompt.txt`
2. calls Gemini and receives a Python file
3. writes the file to disk
4. runs `unittest`
5. if tests fail, appends the error to the prompt and tries again, up to a cap

What we are building

A Python script that:

1. reads a task description from `prompt.txt`
2. calls Gemini and receives a Python file
3. writes the file to disk
4. runs `unittest`
5. if tests fail, appends the error to the prompt and tries again, up to a cap

What we are building

A Python script that:

1. reads a task description from `prompt.txt`
2. calls Gemini and receives a Python file
3. writes the file to disk
4. runs `unittest`
5. if tests fail, appends the error to the prompt and tries again, up to a cap

This is **Pattern 3** (programmatic access) extended with test feedback.

Four files, two roles:

Four files, two roles:

`iterating_regression/`

`prompt.txt`

← we write this

`tests/`

`test_regression.py`

← we write this

`regression.py`

← Gemini writes this

`iterating_regression.py`

← we write this (the driver)

Four files, two roles:

`iterating_regression/`

`prompt.txt` ← we write this

`tests/`

`test_regression.py` ← we write this

`regression.py` ← Gemini writes this

`iterating_regression.py` ← we write this (the driver)

The test file and the prompt define the contract.

Four files, two roles:

`iterating_regression/`

`prompt.txt` ← we write this

`tests/`

`test_regression.py` ← we write this

`regression.py` ← Gemini writes this

`iterating_regression.py` ← we write this (the driver)

The test file and the prompt define the contract.

The driver runs the loop.

The test file

```
import unittest

import numpy as np

from regression import fit_regression

class TestFitRegression(unittest.TestCase):
    def setUp(self):
        self.X = np.array([[0], [1], [2], [3]], dtype=float)
        self.y = np.array([1, 3, 5, 7], dtype=float)

    def test_slope(self):
        beta = fit_regression(self.X, self.y)
        # Testing against all cases (beta[1] = 2.0, y-intercept = 1.0)
```

```
... def test_intercept(self):  
    beta = fit_regression(self.X, self.y)  
    np.testing.assert_allclose(beta[0], 1.0, atol=1e-8)  
  
def test_shape(self):  
    beta = fit_regression(self.X, self.y)  
    self.assertEqual(len(beta), 2)  
  
def test_add_intercept_false(self):  
    X2 = np.hstack([np.ones((4, 1)), self.X])  
    beta = fit_regression(X2, self.y, add_intercept=False)  
    np.testing.assert_allclose(beta, [1.0, 2.0], atol=1e-8)
```

`test_add_intercept_false` passes a design matrix that already has an intercept column and sets `add_intercept=False`.

`test_add_intercept_false` passes a design matrix that already has an intercept column and sets `add_intercept=False`.

A naive implementation that always prepends ones produces:

Why the fourth test

`test_add_intercept_false` passes a design matrix that already has an intercept column and sets `add_intercept=False`.

A naive implementation that always prepends ones produces:

```
X =  [[1, 0],    →  [[1, 1, 0],  
      [1, 1],      [1, 1, 1],  
      [1, 2],      [1, 1, 2],  
      [1, 3]]      [1, 1, 3]]
```


Why the fourth test

`test_add_intercept_false` passes a design matrix that already has an intercept column and sets `add_intercept=False`.

A naive implementation that always prepends ones produces:

```
X = [[1, 0],    → [[1, 1, 0],  
      [1, 1],      [1, 1, 1],  
      [1, 2],      [1, 1, 2],  
      [1, 3]]      [1, 1, 3]]
```

The first two columns are identical: the matrix $X^T @ X$ is singular.

Why the fourth test

`test_add_intercept_false` passes a design matrix that already has an intercept column and sets `add_intercept=False`.

A naive implementation that always prepends ones produces:

```
X =  [[1, 0],    →  [[1, 1, 0],  
      [1, 1],      [1, 1, 1],  
      [1, 2],      [1, 1, 2],  
      [1, 3]]      [1, 1, 3]]
```

The first two columns are identical: the matrix $X^T @ X$ is singular.

The test exists to force the model to handle the `add_intercept=False` path correctly.

Implement a function `fit_regression(X, y, add_intercept=True)`
in `regression.py`.

- When `add_intercept=True`, prepend a column of ones to `X` before solving.
- Return a 1-D array of coefficients `[intercept, slope, ...]`.
- Use `numpy` only. Do not import `sklearn`.
- Do not modify the test file.

The prompt

Implement a function `fit_regression(X, y, add_intercept=True)` in `regression.py`.

- When `add_intercept=True`, prepend a column of ones to `X` before solving.
- Return a 1-D array of coefficients `[intercept, slope, ...]`.
- Use `numpy` only. Do not import `sklearn`.
- Do not modify the test file.

Four elements worth noting:

The prompt

Implement a function `fit_regression(X, y, add_intercept=True)` in `regression.py`.

- When `add_intercept=True`, prepend a column of ones to `X` before solving.
- Return a 1-D array of coefficients `[intercept, slope, ...]`.
- Use `numpy` only. Do not import `sklearn`.
- Do not modify the test file.

Four elements worth noting:

- names the file (`regression.py`) and the function signature

The prompt

Implement a function `fit_regression(X, y, add_intercept=True)` in `regression.py`.

- When `add_intercept=True`, prepend a column of ones to `X` before solving.
- Return a 1-D array of coefficients `[intercept, slope, ...]`.
- Use `numpy` only. Do not import `sklearn`.
- Do not modify the test file.

Four elements worth noting:

- names the file (`regression.py`) and the function signature
- specifies **behavior** for each flag value

The prompt

Implement a function `fit_regression(X, y, add_intercept=True)`
in `regression.py`.

- When `add_intercept=True`, prepend a column of ones to `X` before solving.
- Return a 1-D array of coefficients `[intercept, slope, ...]`.
- Use `numpy` only. Do not import `sklearn`.
- Do not modify the test file.

Four elements worth noting:

- names the **file** (`regression.py`) and the **function signature**
- specifies **behavior** for each flag value
- names the **allowed dependency**

The prompt

Implement a function `fit_regression(X, y, add_intercept=True)`
in `regression.py`.

- When `add_intercept=True`, prepend a column of ones to `X` before solving.
- Return a 1-D array of coefficients `[intercept, slope, ...]`.
- Use `numpy` only. Do not import `sklearn`.
- Do not modify the test file.

Four elements worth noting:

- names the file (`regression.py`) and the function signature
- specifies **behavior** for each flag value
- names the **allowed dependency**
- forbids **test modification**

Creating the client

```
from gemini_simple_api import GeminiSimpleAPI

client = GeminiSimpleAPI(
    api_key_file=Path(".gemini-key"),
    model="gemini-2.5-flash-lite",
    working_dir=TASK_DIR,
)
```

Creating the client

```
from gemini_simple_api import GeminiSimpleAPI
```

```
client = GeminiSimpleAPI(  
    api_key_file=Path(".gemini-key"),  
    model="gemini-2.5-flash-lite",  
    working_dir=TASK_DIR,  
)
```

Three parameters:

Creating the client

```
from gemini_simple_api import GeminiSimpleAPI
```

```
client = GeminiSimpleAPI(  
    api_key_file=Path(".gemini-key"),  
    model="gemini-2.5-flash-lite",  
    working_dir=TASK_DIR,  
)
```

Three parameters:

- `api_key_file` — path to `{"api_key": "..."} JSON`, or ignored if `GEMINI_API_KEY` is set in the environment

Creating the client

```
from gemini_simple_api import GeminiSimpleAPI
```

```
client = GeminiSimpleAPI(  
    api_key_file=Path(".gemini-key"),  
    model="gemini-2.5-flash-lite",  
    working_dir=TASK_DIR,  
)
```

Three parameters:

- `api_key_file` — path to `{"api_key": "..."} JSON`, or ignored if `GEMINI_API_KEY` is set in the environment
- `model` — any name from `GeminiSimpleAPI.MODEL_LIST`

Creating the client

```
from gemini_simple_api import GeminiSimpleAPI
```

```
client = GeminiSimpleAPI(  
    api_key_file=Path(".gemini-key"),  
    model="gemini-2.5-flash-lite",  
    working_dir=TASK_DIR,  
)
```

Three parameters:

- `api_key_file` — path to `{"api_key": "..."} JSON`, or ignored if `GEMINI_API_KEY` is set in the environment
- `model` — any name from `GeminiSimpleAPI.MODEL_LIST`
- `working_dir` — where generated files will be written

```
files, notes = client.prompt(  
    prompt=prompt_text,  
    attachments=[TEST_FILE],  
    verbose=True,  
)
```

Calling the model

```
files, notes = client.prompt(  
    prompt=prompt_text,  
    attachments=[TEST_FILE],  
    verbose=True,  
)
```

`client.prompt()` does three things in one call:

Calling the model

```
files, notes = client.prompt(  
    prompt=prompt_text,  
    attachments=[TEST_FILE],  
    verbose=True,  
)
```

`client.prompt()` does three things in one call:

1. calls `generate_content_structured` — POST to Gemini, get back JSON

Calling the model

```
files, notes = client.prompt(  
    prompt=prompt_text,  
    attachments=[TEST_FILE],  
    verbose=True,  
)
```

`client.prompt()` does three things in one call:

1. calls `generate_content_structured` — POST to Gemini, get back JSON
2. calls `write_files` — writes each file under `working_dir`

Calling the model

```
files, notes = client.prompt(  
    prompt=prompt_text,  
    attachments=[TEST_FILE],  
    verbose=True,  
)
```

`client.prompt()` does three things in one call:

1. calls `generate_content_structured` — POST to Gemini, get back JSON
2. calls `write_files` — writes each file under `working_dir`
3. returns `(list[Path], notes_string)`

Calling the model

```
files, notes = client.prompt(  
    prompt=prompt_text,  
    attachments=[TEST_FILE],  
    verbose=True,  
)
```

`client.prompt()` does three things in one call:

1. calls `generate_content_structured` — POST to Gemini, get back JSON
2. calls `write_files` — writes each file under `working_dir`
3. returns `(list[Path], notes_string)`

Calling the model

```
files, notes = client.prompt(  
    prompt=prompt_text,  
    attachments=[TEST_FILE],  
    verbose=True,  
)
```

`client.prompt()` does three things in one call:

1. calls `generate_content_structured` — POST to Gemini, get back JSON
2. calls `write_files` — writes each file under `working_dir`
3. returns `(list[Path], notes_string)`

`attachments` sends `test_regression.py` as context so the model can see the exact function signature and assertions it must satisfy.

Why attach the test file?

The test file **is** the specification. Attaching it means the model reads the actual contract:

Why attach the test file?

The test file **is** the specification. Attaching it means the model reads the actual contract:

- exact function name and signature (`fit_regression(X, y, add_intercept=True)`)

Why attach the test file?

The test file **is** the specification. Attaching it means the model reads the actual contract:

- exact function name and signature (`fit_regression(X, y, add_intercept=True)`)
- the file it must write (`regression.py` — visible from the import line)

Why attach the test file?

The test file **is** the specification. Attaching it means the model reads the actual contract:

- exact function name and signature (`fit_regression(X, y, add_intercept=True)`)
- the file it must write (`regression.py` — visible from the import line)
- the return shape (1-D array, indexed as `beta[0]`, `beta[1]`)

Why attach the test file?

The test file **is** the specification. Attaching it means the model reads the actual contract:

- exact function name and signature (`fit_regression(X, y, add_intercept=True)`)
- the file it must write (`regression.py` — visible from the import line)
- the return shape (1-D array, indexed as `beta[0]`, `beta[1]`)
- the behavior of `add_intercept=False` (made concrete by the fourth test)

Why attach the test file?

The test file **is** the specification. Attaching it means the model reads the actual contract:

- exact function name and signature (`fit_regression(X, y, add_intercept=True)`)
- the file it must write (`regression.py` — visible from the import line)
- the return shape (1-D array, indexed as `beta[0]`, `beta[1]`)
- the behavior of `add_intercept=False` (made concrete by the fourth test)

Why attach the test file?

The test file **is** the specification. Attaching it means the model reads the actual contract:

- exact function name and signature (`fit_regression(X, y, add_intercept=True)`)
- the file it must write (`regression.py` — visible from the import line)
- the return shape (1-D array, indexed as `beta[0]`, `beta[1]`)
- the behavior of `add_intercept=False` (made concrete by the fourth test)

You could repeat all of this in `prompt.txt`, but prose summaries are easier to misread than executable code.

Caveats: attaching the test file

The model can see paths it could overwrite.

Caveats: attaching the test file

The model can see paths it could overwrite.

`working_dir` and the test directory are the same folder here. The model is told “do not modify the test file” — but that is a soft constraint. It **could** propose overwriting `tests/test_regression.py`.

Caveats: attaching the test file

The model can see paths it could overwrite.

`working_dir` and the test directory are the same folder here. The model is told “do not modify the test file” — but that is a soft constraint. It **could** propose overwriting `tests/test_regression.py`.

Mitigations:

Caveats: attaching the test file

The model can see paths it could overwrite.

`working_dir` and the test directory are the same folder here. The model is told “do not modify the test file” — but that is a soft constraint. It **could** propose overwriting `tests/test_regression.py`.

Mitigations:

- keep tests outside `working_dir` (separate directory)

Caveats: attaching the test file

The model can see paths it could overwrite.

`working_dir` and the test directory are the same folder here. The model is told “do not modify the test file” — but that is a soft constraint. It **could** propose overwriting `tests/test_regression.py`.

Mitigations:

- keep tests outside `working_dir` (separate directory)
- use a read-only bind mount in Docker (kernel-enforced)

Caveats: attaching the test file

The model can see paths it could overwrite.

`working_dir` and the test directory are the same folder here. The model is told “do not modify the test file” — but that is a soft constraint. It **could** propose overwriting `tests/test_regression.py`.

Mitigations:

- keep tests outside `working_dir` (separate directory)
- use a read-only bind mount in Docker (kernel-enforced)
- check `git diff tests/` before accepting any patch

Caveats: attaching the test file

The model can see paths it could overwrite.

`working_dir` and the test directory are the same folder here. The model is told “do not modify the test file” — but that is a soft constraint. It **could** propose overwriting `tests/test_regression.py`.

Mitigations:

- keep tests outside `working_dir` (separate directory)
- use a read-only bind mount in Docker (kernel-enforced)
- check `git diff tests/` before accepting any patch
- squirrel the test file away somewhere and copy it back into the working directory after the model is done

A model can make tests pass without solving the scientific problem.

A model can make tests pass without solving the scientific problem.

Attaching tests gives it enough information to hardcode expected values or weaken assertions. For a simple numerical function this is unlikely and easy to spot; for complex logic it is a real risk.

A model can make tests pass without solving the scientific problem.

Attaching tests gives it enough information to hardcode expected values or weaken assertions. For a simple numerical function this is unlikely and easy to spot; for complex logic it is a real risk.

You should own at least one side of the ledger — either you write the tests, or you review AI-generated tests very closely.

A model can make tests pass without solving the scientific problem.

Attaching tests gives it enough information to hardcode expected values or weaken assertions. For a simple numerical function this is unlikely and easy to spot; for complex logic it is a real risk.

You should own at least one side of the ledger — either you write the tests, or you review AI-generated tests very closely.

Or you can of course feed it only part of the test suite and keep a secret part for validation.

`generate_content_structured` enforces a JSON schema on the response:

What the model returns

`generate_content_structured` enforces a JSON schema on the response:

```
{
  "files": [
    {
      "name": "regression.py",
      "relative_path": "regression.py",
      "content": "import numpy as np\n\ndef fit_regression(X, y, ...)"
    }
  ],
  "notes": "Used np.linalg.solve with an intercept guard."
}
```


What the model returns

`generate_content_structured` enforces a JSON schema on the response:

```
{
  "files": [
    {
      "name": "regression.py",
      "relative_path": "regression.py",
      "content": "import numpy as np\n\ndef fit_regression(X, y, ..."
```

The model cannot respond with free text — it must produce valid JSON matching this schema.

Running the tests

```
def run_tests() -> tuple[int, str]:  
    result = subprocess.run(  
        ["python", "-m", "unittest", "discover", "-s", "tests"],  
        cwd=TASK_DIR,  
        capture_output=True,  
        text=True,  
    )  
    return result.returncode, (result.stdout + result.stderr).strip()
```

Running the tests

```
def run_tests() -> tuple[int, str]:  
    result = subprocess.run(  
        ["python", "-m", "unittest", "discover", "-s", "tests"],  
        cwd=TASK_DIR,  
        capture_output=True,  
        text=True,  
    )  
    return result.returncode, (result.stdout + result.stderr).strip()
```

returncode is 0 on success, non-zero on failure.

Running the tests

```
def run_tests() -> tuple[int, str]:  
    result = subprocess.run(  
        ["python", "-m", "unittest", "discover", "-s", "tests"],  
        cwd=TASK_DIR,  
        capture_output=True,  
        text=True,  
    )  
    return result.returncode, (result.stdout + result.stderr).strip()
```

`returncode` is 0 on success, non-zero on failure.

`stdout + stderr` captures the full unittest output including tracebacks.

Running the tests

```
def run_tests() -> tuple[int, str]:  
    result = subprocess.run(  
        ["python", "-m", "unittest", "discover", "-s", "tests"],  
        cwd=TASK_DIR,  
        capture_output=True,  
        text=True,  
    )  
    return result.returncode, (result.stdout + result.stderr).strip()
```

`returncode` is 0 on success, non-zero on failure.

`stdout + stderr` captures the full unittest output including tracebacks.

The exit code tells us whether to stop; the output tells the model what went wrong.

Attempt 1: failure

Wrote 1 files:

- + .../regression.py

```
test_add_intercept_false ... ERROR  
[... 3 other tests pass]
```

```
ERROR: test_add_intercept_false  
LinAlgError: Singular matrix  
Ran 4 tests in 0.012s  
FAILED (errors=1)
```

Attempt 1: failure

Wrote 1 files:

- + .../regression.py

```
test_add_intercept_false ... ERROR
[... 3 other tests pass]
```

```
ERROR: test_add_intercept_false
LinAlgError: Singular matrix
Ran 4 tests in 0.012s
FAILED (errors=1)
```

Three tests pass; one fails — exactly the one that exposed the missing guard.

Building the feedback prompt

```
prompt_text += (  
    f"\n\n## Attempt {attempt} failed\n"  
    f"```\n{output}\n```\n"  
    "Fix the failures above."  
)
```


Building the feedback prompt

```
prompt_text += (  
    f"\n\n## Attempt {attempt} failed\n"  
    f"```\n{output}\n```\n"  
    "Fix the failures above."  
)
```

The next call sends:

Building the feedback prompt

```
prompt_text += (  
    f"\n\n## Attempt {attempt} failed\n"  
    f"```\n{output}\n```\n"  
    "Fix the failures above."  
)
```

The next call sends:

- the original task (function name, file name, constraints)

Building the feedback prompt

```
prompt_text += (  
    f"\n\n## Attempt {attempt} failed\n"  
    f"```\n{output}\n```\n"  
    "Fix the failures above."  
)
```

The next call sends:

- the original task (function name, file name, constraints)
- the test file (still attached as **attachments**)

Building the feedback prompt

```
prompt_text += (  
    f"\n\n## Attempt {attempt} failed\n"  
    f"```\n{output}\n```\n"  
    "Fix the failures above."  
)
```

The next call sends:

- the original task (function name, file name, constraints)
- the test file (still attached as **attachments**)
- the exact test output from the failed run

Building the feedback prompt

```
prompt_text += (  
    f"\n\n## Attempt {attempt} failed\n"  
    f"```\n{output}\n```\n"  
    "Fix the failures above."  
)
```

The next call sends:

- the original task (function name, file name, constraints)
- the test file (still attached as **attachments**)
- the exact test output from the failed run

Building the feedback prompt

```
prompt_text += (  
    f"\n\n## Attempt {attempt} failed\n"  
    f"```\n{output}\n```\n"  
    "Fix the failures above."  
)
```

The next call sends:

- the original task (function name, file name, constraints)
- the test file (still attached as **attachments**)
- the exact test output from the failed run

The model now has everything it needs: the contract it failed and the error it produced.

Attempt 2: passes

```
=== Attempt 2 ===
```

```
Wrote 1 files:
```

```
  + .../regression.py
```

```
test_add_intercept_false ... ok
```

```
test_intercept ... ok
```

```
test_shape ... ok
```

```
test_slope ... ok
```

```
Ran 4 tests in 0.009s
```

```
OK
```

```
Tests passed on attempt 2.
```

The corrected implementation added the guard:

The corrected implementation added the guard:

```
if add_intercept:  
    X = np.hstack([np.ones((X.shape[0], 1)), X])
```

The full driver

```
prompt_text = (TASK_DIR / "prompt.txt").read_text()
for attempt in range(1, MAX_ATTEMPTS + 1):
    print(f"\n=== Attempt {attempt} ===")
    files, notes = client.prompt(
        prompt=prompt_text, attachments=[TEST_FILE], verbose=True,
    )
    code, output = run_tests()
    print(output)
    if code == 0:
        print(f"\nTests passed on attempt {attempt}.")
        break
    prompt_text += (
        f"\n\n## Attempt {attempt} failed\n"
        f"```\n{output}\n```\nFix the failures above."
    )
else: sys.exit(1)
```

The loop is ~15 lines. The feedback accumulates in `prompt_text` — each round the model sees the full history of failures.

The loop is ~15 lines. The feedback accumulates in `prompt_text` — each round the model sees the full history of failures.

Full script: `files/iterating_regression/iterating_regression.py`

What structured output buys you

Without the schema, the model returns something like:

What structured output buys you

Without the schema, the model returns something like:

Here is the implementation: [code block]

Let me know if you need changes!

What structured output buys you

Without the schema, the model returns something like:

Here is the implementation: [code block]

Let me know if you need changes!

You would need to:

What structured output buys you

Without the schema, the model returns something like:

Here is the implementation: [code block]

Let me know if you need changes!

You would need to:

- strip the explanation text

What structured output buys you

Without the schema, the model returns something like:

Here is the implementation: [code block]

Let me know if you need changes!

You would need to:

- strip the explanation text
- find the code block

What structured output buys you

Without the schema, the model returns something like:

Here is the implementation: [code block]

Let me know if you need changes!

You would need to:

- strip the explanation text
- find the code block
- figure out what file to write it to

What structured output buys you

Without the schema, the model returns something like:

Here is the implementation: [code block]

Let me know if you need changes!

You would need to:

- strip the explanation text
- find the code block
- figure out what file to write it to

What structured output buys you

Without the schema, the model returns something like:

Here is the implementation: [code block]

Let me know if you need changes!

You would need to:

- strip the explanation text
- find the code block
- figure out what file to write it to

With `generate_content_structured`, you get a dict with `files[0]["content"]` and `files[0]["relative_path"]` — write and continue.

From the previous lecture:

Pattern 3, applied

From the previous lecture:

Pattern	Typical use
Single-turn generation	Draft one function
Context-aware completion	Editor autocomplete
Programmatic access	Call model from code
Autonomous execution	Agent with tools

Pattern 3, applied

From the previous lecture:

Pattern	Typical use
Single-turn generation	Draft one function
Context-aware completion	Editor autocomplete
Programmatic access	Call model from code
Autonomous execution	Agent with tools

This script is Pattern 3 with a hand-written feedback loop: no agent framework, no special tools, just Python and a while loop.

Pattern 3, applied

From the previous lecture:

Pattern	Typical use
Single-turn generation	Draft one function
Context-aware completion	Editor autocomplete
Programmatic access	Call model from code
Autonomous execution	Agent with tools

This script is Pattern 3 with a hand-written feedback loop: no agent framework, no special tools, just Python and a while loop.

The loop adds one thing: **test output as context**. That is enough to fix a class of simple mistakes.

Tests pass — but that is not the same as correct.

The limits

Tests pass — but that is not the same as correct.

Things a passing test suite does **not** guarantee:

Tests pass — but that is not the same as correct.

Things a passing test suite does **not** guarantee:

- numerical stability on ill-conditioned inputs

Tests pass — but that is not the same as correct.

Things a passing test suite does **not** guarantee:

- numerical stability on ill-conditioned inputs
- correct behavior outside the tested range

Tests pass — but that is not the same as correct.

Things a passing test suite does **not** guarantee:

- numerical stability on ill-conditioned inputs
- correct behavior outside the tested range
- absence of silent approximations

Tests pass — but that is not the same as correct.

Things a passing test suite does **not** guarantee:

- numerical stability on ill-conditioned inputs
- correct behavior outside the tested range
- absence of silent approximations
- a formula the author can explain

Tests pass — but that is not the same as correct.

Things a passing test suite does **not** guarantee:

- numerical stability on ill-conditioned inputs
- correct behavior outside the tested range
- absence of silent approximations
- a formula the author can explain

Tests pass — but that is not the same as correct.

Things a passing test suite does **not** guarantee:

- numerical stability on ill-conditioned inputs
- correct behavior outside the tested range
- absence of silent approximations
- a formula the author can explain

The tests are the **contract**. The model satisfies the contract.

Tests pass — but that is not the same as correct.

Things a passing test suite does **not** guarantee:

- numerical stability on ill-conditioned inputs
- correct behavior outside the tested range
- absence of silent approximations
- a formula the author can explain

The tests are the **contract**. The model satisfies the contract.

Whether the contract was strong enough is still your call.